Performance Analysis of Algorithms

Why Performance Analysis?

- There are many important aspects of an algorithm/application that should be taken care of, like user-friendliness, modularity, security, maintainability, etc. Why worry about performance?
- The performance of an algorithm can change with a change in the input size.
- Performance of an algorithm may not matter much for small input size, but when the input size increases, performance can not be ignored.
- Many real-time and mission-critical applications might result in havoc if results are not produced within some specific time.
- Algorithms should be optimized.

What are Asymptotic Notations?

- Asymptotic Notations are languages that allow us to analyze an algorithm's run-time performance.
- Asymptotic Notations identify running time by algorithm behavior as the input size for the algorithm increases.
- This is also known as an algorithm's growth rate. Usually, the time required by an algorithm falls under three types −
  - **Best Case** − Minimum time required for program execution
  - **Average Case** − Average time required for program execution.
  - **Worst Case** − Maximum time required for program execution.

Types of Asymptotic Notations

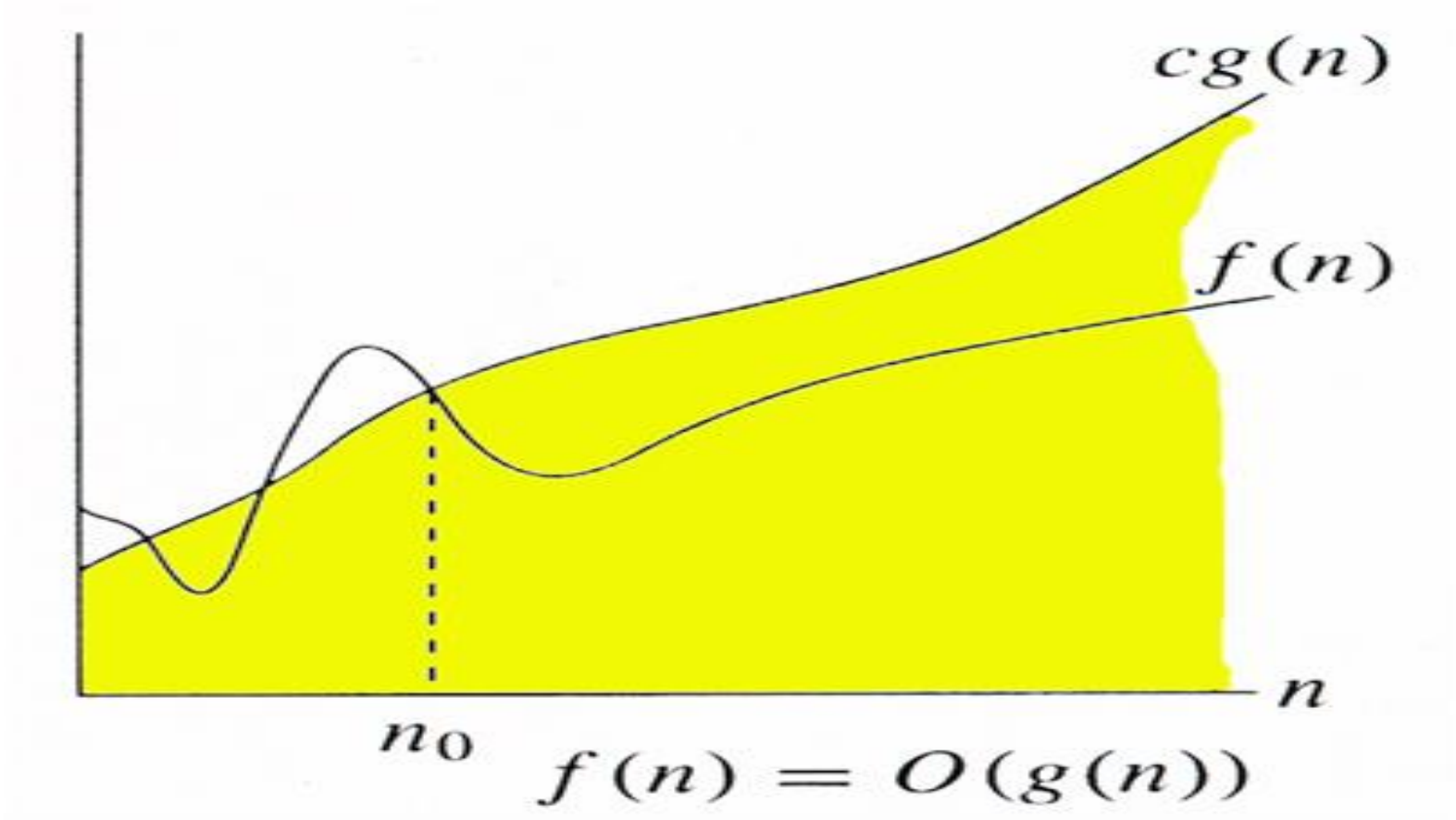
- Commonly used asymptotic notations to calculate the running time complexity of an algorithm.
  - O Notation (Big-O Notation)
  - Ω Notation (Big-Omega Notation)
  - θ Notation (Theta Notation)

O Notation (Big-O Notation)

- Big-O, commonly written as O, is an Asymptotic Notation for the worst case or the longest time an algorithm can take to complete.
- It describes the upper bound of an algorithm's runtime and calculates the time and amount of memory needed to execute the algorithm for an input value.
- Upper bound for a function can be defined as
- Let f(n) and g(n) are two nonnegative functions indicating the running time of two algorithms. We say, g(n) is upper bound of f(n) if there exist some positive constants c and n_0 such that $0 \leq f(n) \leq c.g(n)$ for all $n \geq n_0$.
- It is denoted as f(n) = O(g(n)).

For further details, refer: https://codecrucks.com/examples-of-asymptotic-notation/

O Notation (Big-O Notation)

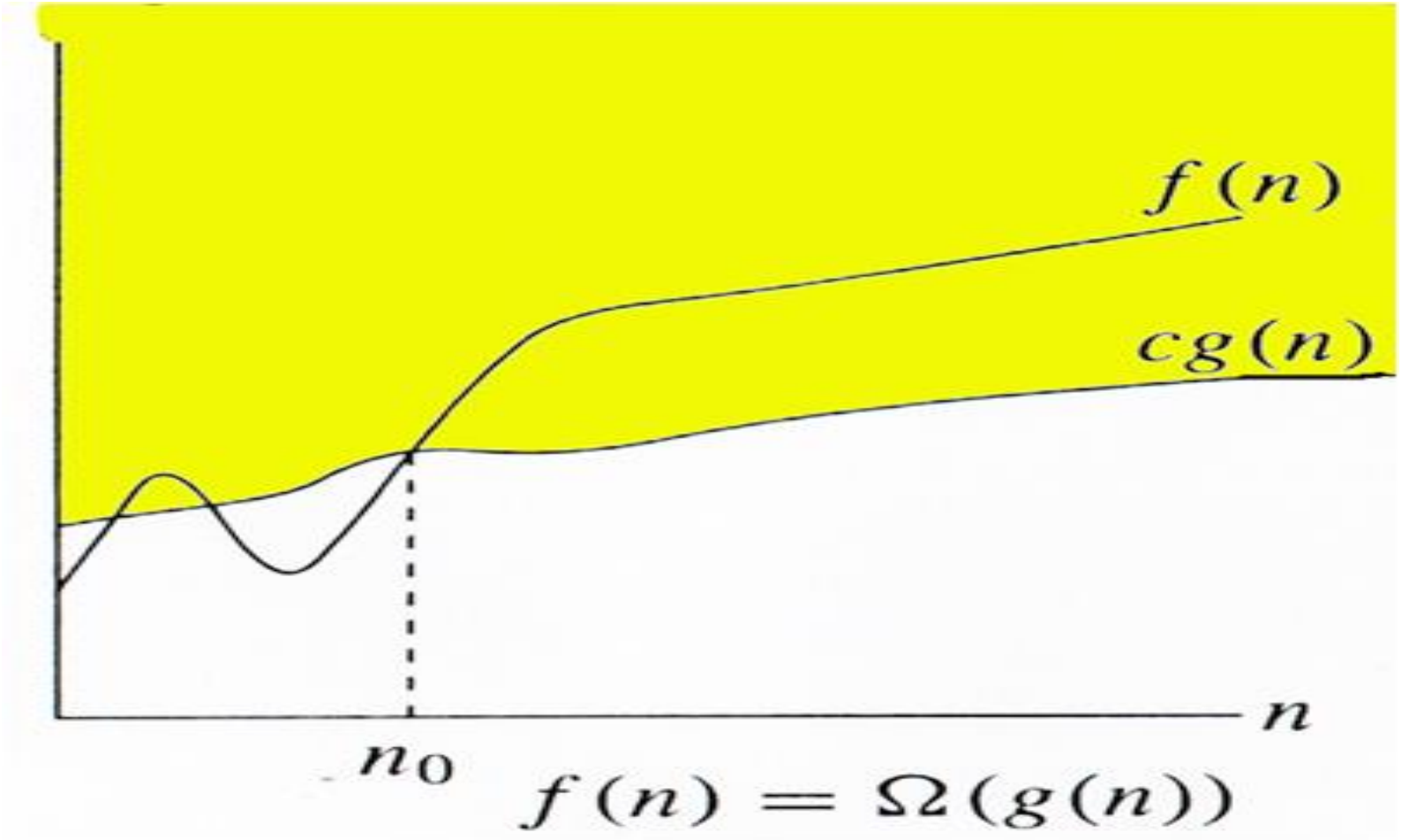


Ω Notation (Big-Omega Notation)

- Big-Omega, commonly written as Ω, is an Asymptotic Notation for the best case, or the best amount of time an algorithm can possibly take to complete.
- It provides us with an asymptotic lower bound for the growth rate of run-time of an algorithm.
- **For example**, for a function f(n)

$\Omega(f(n)) \geq \{ g(n)$: there exists $c > 0$ and $n0$ such that $c.g(n) \leq f(n)$ for all $n > n0$. $\}$

Ω Notation (Big-Omega Notation)



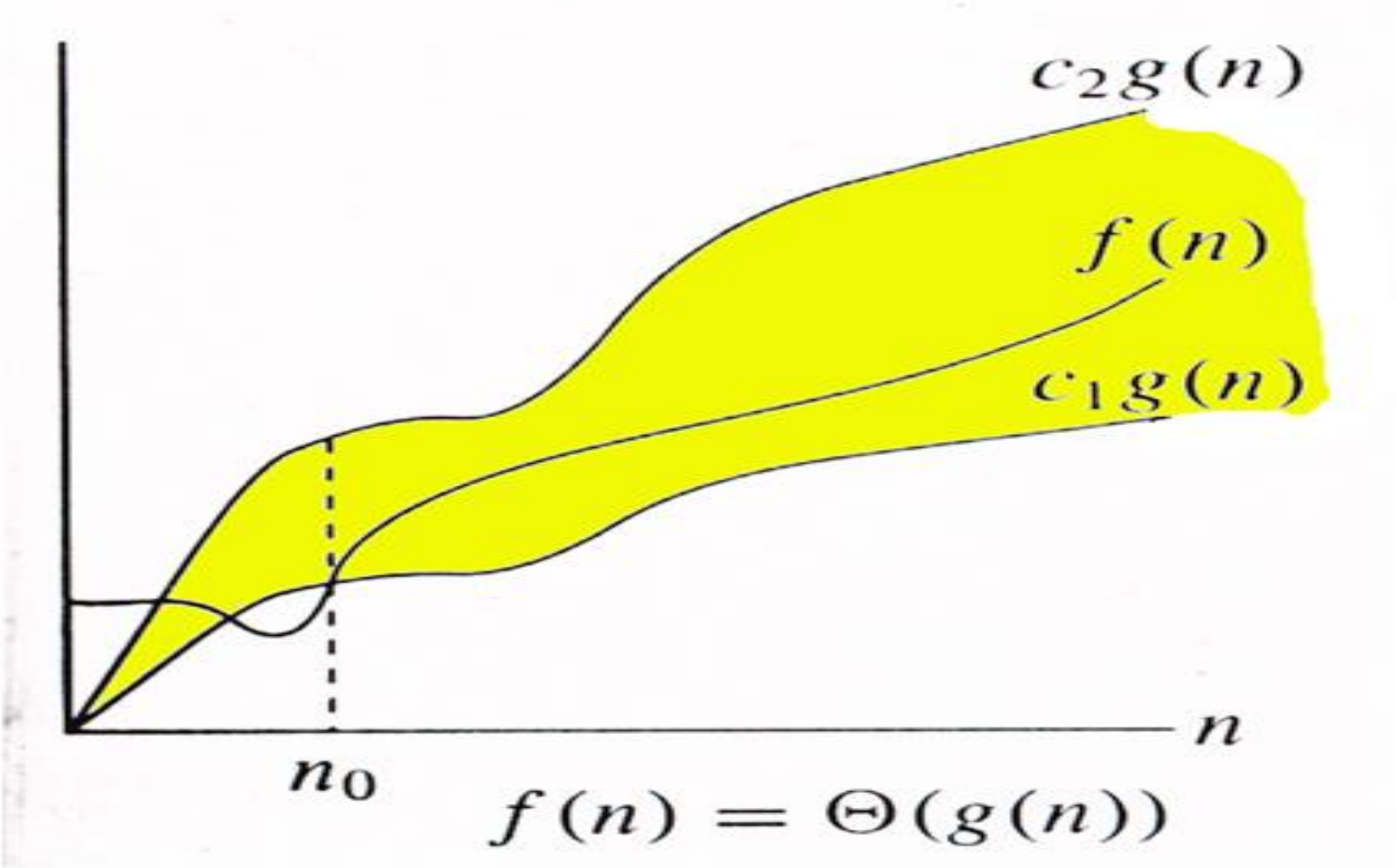
θ Notation (Theta Notation)

- Theta, commonly written as Θ, is an Asymptotic Notation to denote the asymptotically tight bound (lower bound and the upper bound) on the growth rate of run-time of an algorithm.
- **For example**, for a function f(n)

θ(f(n)) = { g(n) if and only if g(n) = O(f(n)) and g(n) = Ω(f(n)) for all

n > n0. }

θ Notation (Theta Notation)



Common Asymptotic Notations

- Following is a list of some common asymptotic notations –

NOTATION	NAME
O(1)	constant
O(log n)	logarithmic
O(n)	linear
O(n log n) = O(log n!)	linearithmic, loglinear, or quasilinear
$O(n^2)$	quadratic
$O(n^c)$	polynomial or algebraic
$O(c^n)$; where c>1	exponential
O(n!)	factorial

Common Asymptotic Notations

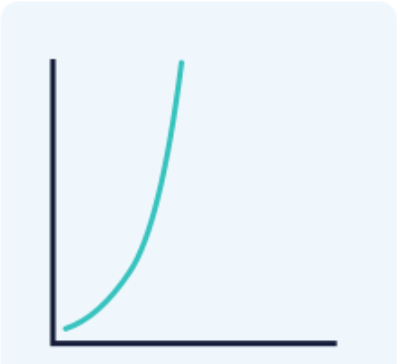
$\Theta(1)$

$\Theta(\log N)$

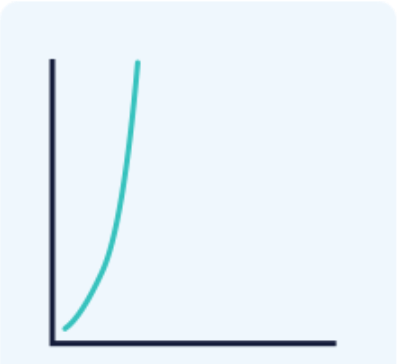
$\Theta(N)$

$\Theta(N \log N)$

$\Theta(N^2)$



$\Theta(2^N)$



$\Theta(N!)$

Complexity of an Algorithm

Rules for Order Arithmetic:

- Multiplicative Constants
  $O(k * f(n)) = O(f(n))$

- Addition Rule
  $O(f(n) + g(n)) = \max(f(n), g(n))$

- Multiplication Rule
  $O(f(n) * g(n)) = O(f(n)) * O(g(n))$

- **Examples:**

- $O(1000\ n) = O(n)$

- $O(n^2 + 3n + 2) = O(n^2)$

- $O(3n^3 + 6n^2 - 4n + 2) = O(3n^3) = O(n^3)$

- If $f(x) = n^2 * \log n = O(n^2 \log n)$

Complexity of an Algorithm

```
for (i = 0; i < n; i++)

   for (j = 0; j < n;  j++)

      cin >> A[i][j];
```

Number of times cin >> A[i][j]; **executed: n^2**
Complexity: O (n^2)

In a nested for loop with fixed lower and upper limits (e.g. "0" and "n") the number of times the insider part is executed is just n x n. f(n) = n^2 . So this program section runs in O(n^2) time.

Complexity of an Algorithm

```
for (i = 0; i < n; i++)

  for (j = 0; j < i; j++)

     A[i][j] = 0;
```

The inner loop is executed a variable amount of times; we can't just multiply n by i:

When i = 0 the inner loop is executed 0 times.
When i = 1 the inner loop is executed 1 time.
When i = 2 the inner loop is executed 2 times.
So the number of times the statement "A[i][j] = 0" is executed is: 1 + 2 + 3 + ... + n = $n(n+1)/2 = \frac{1}{2}(n^2+n)$

Ignore leading constants (the $1/2$ in the equation above) and lower order terms (the n in the equation above), this algorithm runs in $O(n^2)$ time.

Complexity of an Algorithm

```
Dim iSum,i,j,k As Integer
   For i = 1 to n
     For j = 1 to n
       iSum = iSum + 1       |   =  1 x n
     end for
     For k = 1 to 2 *n                          =   x  n
       iSum = iSum + 1       |
       iSum = iSum + 1       |   =  3 x 2n
       iSum = iSum + 1       |
     end for
   end for
```

Number of times executed: n x (1 x n + 3 x 2n) = $n^2 + 6n^2 = 7n^2$
Complexity: O (n^2)

Complexity of an Algorithm

```
Dim iSum,i,j,k As Integer
    For i = 1 to n
        For j = 1 to n
            iSum = iSum + 1                     = n
        end for
        For k = 1 to 2 *n                                   = n
            iSum = iSum + 1             = 1
            For m = 1 to 2 * n                  = 2n
                iSum = iSum + 1         = 2n
            end for
        end for
    end for
```

Number of times executed: n x (n + 2n (1+2n)) = $n^2 + 2n^2 + 4n^3$

Complexity: O (n^3)

Complexity of an Algorithm

```
void main(void)
{
    int i, tofind, A[100], n;
    i = 0;
    cin >> tofind;
    while ((A[i] != tofind) && (i <= n))
         i++;
    if (i >= n) cout << "not found";
    else cout << "found";
}
```

Complexity: O (n)

Complexity of an Algorithm

```
1.  i = n;              // i starts from n
2.  while (i >= 1)
3.  {
4.    x = x + 1;        // count this line
5.    i = i / 2;        // i becomes half at the end of every iteration
6.  }
```

iteration	value of i (at the top of loop)	number of times line 4 is executed
1	n	1
2	$n/2^1$	1
3	$n/2^2$	1
k	$n/2^{k-1} = 1$	1
k+1	$n/2^k = 0$	0 (loop not executed)

Complexity: O ($\log_2$ n)

Knowing the complexity in competitive programming

- The most difficult task faced is to write code under given complexity otherwise the program will get a TLE (**Time Limit Exceeded**)
- A naive solution is almost never accepted. So how to know, what complexity is acceptable?
- Most of the platforms support 10^8 operations per second and you are expected to write to code which runs in 1 second.
- So ensure the complexity of your code does not lead to 10^8 operations

Knowing the complexity in competitive programming

- If the complexity of your code is $O(n^2)$ and the constraints

  1) $1 <= N <= 10^3$ 2) $1 <= N <= 10^5$ 3) $1 <= N <= 10^8$
- The solution will work for 1st constraint while for next two it will result in TLE
- For second case, $O(n^2)$ will lead to 10^{10} operations. $O(n\log_2 n)$ - 10^6 or $O(n)$ – 10^5 which is well under 10^8
- For third case, even $O(n \log_2 n)$ will lead to approx 10^9 operations. Only acceptable solution is $O(n)$
- Never ignore the constraints…… Plan a solution as per the given constraint.

Acceptable solution for Input Sizes

Input size(n)	Time complexity
$n \leq 10$	O(n!)
$n \leq 50$	$O(2^n)$
$n \leq 500$	$O(n^3)$
$n \leq 5000$	$O(n^2)$
$n \leq 10^6$	O(n logn) or O(n)
n is very large	O(1) or O(logn)

Example Problem

Given an array of n numbers, the task is to calculate the maximum subarray sum. The largest possible sum of a sequence of consecutive values in the array.

- The obvious solution is $O(n^3)$ but other approaches are possible with better performance

Example Problem – Approach 1

```
int subarray_sum(int nums[], int n)
{
    int ans = 0;
    for(int i=0;i<n;i++){
        for(int j=i;j<n;j++){
            int sum=0;
            for(int k=i;k<=j;k++){
                sum+= nums[k];
            }
            ans = max(sum, ans);
        }
    }
    return ans;
}
```

Complexity $O(n^3)$

Example Problem – Approach 2

```
int subarray_sum2(int nums[], int n)
{
    int ans=0;
    for(int i=0;i<n;i++){
        int sum=0;
        for(int j=i;j<n;j++){
            sum+= nums[j];
            ans = max(sum,ans);
        }
    }
    return ans;
}
```

Complexity $O(n^2)$

Example Problem – Approach 3

```
int subarray_sum3(int nums[], int n)
{
    int ans = 0, sum = 0;
    for(int i=0;i<n;i++)
        {
        sum = max(nums[i], sum+A[i]);
        ans = max(sum,ans);
    }
    return ans;
}
```

Complexity O(n)

Comparison of Approaches

Array size (n)	Method number 1	Method number 2	Method number 3
10^2	0.0 s	0.0 s	0.0 s
10^3	0.1 s	0.0 s	0.0 s
10^4	> 10.0 s	0.1 s	0.0 s
10^5	> 10.0 s	5.0 s	0.0 s
10^6	> 10.0 s	> 10.0 s	0.0 s
10^7	> 10.0 s	> 10.0 s	0.0 s

Source: https://www.geeksforgeeks.org/knowing-the-complexity-in-competitive-programming/